Firefox Domain Trust Extension Design and Evaluation

Recommendation and scope

Implement exclusive domain-to-CA binding as a maintained Firefox source extension, with an optional WebExtension for diagnostics and a separate add-on feasibility experiment. The enforcement component should narrow eligible certificate authorities before certificate-path construction and prevent application traffic until the resulting connection is authorized. This preserves the central objective of the Chromium design while using Firefox's own certificate verifier, network stack, and update mechanisms.

The baseline is the 26 July 2026 companion report, Implementing Exclusive Domain-to-CA Binding in Chromium, together with Constrained Certificate Trust in Chrome, version 2.1. The companion defines ten invariants, 62 implementation work packages, and 50 conformance cases. These are design requirements and planned work; their presence does not establish that the Chrome implementation exists or has passed qualification. The Firefox backlog introduces a separate FDT namespace and retains explicit links to the earlier work.[1]

This report targets desktop Linux and macOS. Start with Linux x86-64 and macOS arm64; qualify Linux arm64 and macOS x86-64 before advertising those architectures. Windows, Android, and iOS are outside the initial qualification. Use a supported Firefox extended support release (ESR) as the proposed product baseline and a current upstream branch for integration reconnaissance. Select and record the exact release, source revision, Network Security Services (NSS) and Neqo revisions, toolchain, platform trust configuration, and patches during intake.

Evidence was checked on 10 September 2026. Public documentation and source inspection support feasibility and identify integration points; this report contains no measured prototype results. Source links that follow moving branches are architectural evidence, not a frozen implementation baseline. All experimental outcomes remain **not run**. Proposed performance budgets are planning targets, not performance claims.

The guarantee inherited from Chrome

For a protected reference hostname such as api.scone.cloud, a successful connection must have a valid certificate path terminating at an explicitly approved CA public key. A normally trusted public CA or an installed interception CA is insufficient unless the applicable rule approves it. Unmatched hosts retain the selected Firefox baseline's ordinary certificate acceptance. Keeping this distinction allows general browsing and enterprise private PKI to coexist with protected services.

CA identity is the SHA-256 digest of DER-encoded SubjectPublicKeyInfo, or SPKI. A label such as ca.scone.cloud, issuer distinguished name, DNS record, or certificate subject is not an authority identifier. A rule may additionally require an approved intermediate in the validated path. Anchor authorization and intermediate requirements are separate predicates; a supplied certificate that is not part of the validated path satisfies neither.

[1] Earlier project reports, 26 July 2026: Extended_Chromium_Custom_Verifier_Companion_Whitepaper.docx, version 1.0, sections 1, 5–10 and Appendices C–D; and Constrained_Certificate_Trust_in_Chrome_Technical_Whitepaper_v2.1.docx. Retrieved project documents; no public URL. The companion is the normative compatibility baseline.

X.509 name constraints limit the names a CA may authenticate; they do not reserve those names against other trusted CAs. Likewise, adding a private root does not remove alternative paths through public or enterprise roots. A policy-supplied CA must retain its applicable constraints, and the new domain restriction must not weaken ordinary certificate checks.[2]

The threat model includes incorrect issuance by unapproved CAs, TLS interception through otherwise trusted roots, malicious network routing, cross-signing, stale connections, replayed signed policies, and hostile web content. The managed browser binary, operating system, bootstrap policy keys, and enforcement component are trusted. Replacing the browser or its bootstrap authority with administrator privileges is outside this guarantee. Approved-CA compromise and erroneous authorized issuance remain substantive residual risks.

In the SCONE setting, this browser feature can ensure that a protected service certificate comes from the governed service CA. It does not itself establish that the endpoint is currently attested, running the newest application policy, or still compliant after issuance. Those properties require explicit CA issuance and revocation contracts. A certificate with a maximum lifetime of three months bounds one exposure interval; it is not continuous attestation or immediate revocation.

Invariant	Firefox requirement	Chrome origin
F1	Unmatched certificate acceptance is baseline-equivalent	I-1
F2	Canonical complete-label matching with authorized child rules	I-2
F3	Filter eligible anchors before path construction	I-3
F4	Preserve explicit distrust and normal validation	I-4
F5	Bind success to origin, rule, policy digest, generation, and anchor	I-5
F6	Apply equivalent policy to TCP TLS and HTTP/3	I-6
F7	Qualify legitimate private PKI over HTTP/3 without unknown-root trust	I-7
F8	Reuse and updates cannot retain obsolete authorization	I-8
F9	Authenticate and atomically activate complete policies	I-9
F10	Rotate authorities through bounded explicit overlap	I-10
F11	No protected application data before current authorization	Added explicit acceptance criterion
F12	Expiry, startup, and missing state cannot erase protected scope	Added explicit acceptance criterion

F1 concerns certificate acceptance. The Chrome companion also demands usable private-PKI HTTP/3, which may require changing a Firefox transport heuristic. Record that as a deliberate transport-availability change if the pinned stock build falls back to HTTP/2. Do not describe it as identical protocol selection.

What a standard Firefox extension can achieve

Firefox exposes `webRequest.getSecurityInfo()` to a blocking `onHeadersReceived` listener. The API can return the certificate chain through its root. This makes a Firefox add-on substantially more useful for inspecting the

[2] IETF, RFC 5280 Internet X.509 Public Key Infrastructure Certificate and CRL Profile, May 2008, sections 4.2.1.10 and 6. Trust anchors are validation inputs; anchor-specific checks must be explicit.

connection than a generic URL-only check.[3] However, onHeadersReceived fires after response headers arrive.[4] The security inference is decisive: the server may already have received the URL path, cookies, authorization headers, and some or all request-body data before an issuer check runs. Cancellation at this stage cannot recover those disclosures or undo server-side effects.

The certificate API already exposes subjectPublicKeyInfoDigest.sha256, a Base64 SHA-256 SPKI digest. Use that field for the observation prototype, with strict decoding and length checks. Do not compare an issuer name, confuse a whole-certificate fingerprint with an SPKI digest, or add a new ASN.1 parser merely to recover an already supplied digest. Missing or ambiguous chain data must yield an unknown or blocked observation, never an approved status.[5]

The WebRequest documentation explicitly limits extensions to reading handshake information rather than changing the browser's trust decisions. It also documents permissions for both resource and initiating-page hosts, plus startup registration requirements. These matter for subresources, service workers, browser startup, and private browsing.[6] The prototype must validate its actual coverage, even though better coverage cannot solve the response-stage timing limit.

A preflight request or native helper probe does not close this gap. The probe may authenticate one connection while Firefox later uses a different connection, address, proxy, certificate, or policy epoch. A correct replacement would need an enforceable binding to the actual browser connection and every request using it. A native-messaging channel alone supplies no such binding.

Approach	Protected first request	Normal Firefox validation	Main use
Enterprise CA installation	Other trusted CAs can still authenticate the host	Retained	CA provisioning
Standard WebExtension	Certificate inspection is too late for the no-disclosure guarantee	Browser controls validation	Observation and feasibility comparison
Separate preflight helper	Probe and browser connection may differ	Depends on implementation	Diagnostics
Mandatory terminating proxy	Possible if every path is enforced	Proxy becomes upstream verifier	Separate architecture with a larger trust boundary
Native Firefox verifier extension	Designed to gate actual authenticated connections	Retained within scoped trust	Recommended equivalent

Firefox enterprise policies provide CA installation, operating-system root import on Windows and macOS, extension installation controls, and certificate-error UI controls. The documented policy surface does not provide the proposed hostname-to-exclusive-anchor mapping. Linux root integration requires its own baseline rather than copying macOS assumptions. Hiding an exception button is also insufficient to establish a non-overridable native error.[7]

[3] Mozilla MDN, webRequest.getSecurityInfo, modified 17 July 2025; accessed 10 September 2026. Callback timing, permissions, and chain option.

[4] Mozilla MDN, webRequest.onHeadersReceived, accessed 10 September 2026. Response-stage event semantics.

[5] Mozilla MDN, webRequest.CertificateInfo, modified 17 July 2025; accessed 10 September 2026. SPKI digests and certificate fingerprints.

[6] Mozilla MDN, webRequest, accessed 10 September 2026. API trust boundary, startup, speculative requests, and host permissions.

[7] Mozilla, Policy Templates for Firefox, accessed 10 September 2026. Certificates, ExtensionSettings, and DisableSecurityBypass sections.

A diagnostic add-on may therefore be delivered independently, but it must say that it reports an observed connection. Only a native, qualified decision may be presented as enforcement. If a companion is distributed for stock Firefox, follow Mozilla's signing and distribution requirements; force installation does not create a certificate-verification capability.[8]

Native architecture

Firefox's certificate-verification stack provides a suitable integration boundary. CertVerifier::VerifySSLServerCert receives the hostname and certificate inputs and returns a built chain; it performs hostname and other TLS-specific checks around general certificate validation.[9] Mozilla's TrustDomain interface supplies GetCertTrust, issuer discovery, and chain-level checks to mozpkix. Its trust levels distinguish an anchor, an actively distrusted certificate, and a certificate that must inherit trust through an issuer.[10]

The proposed components are a privileged policy service, immutable policy snapshots, a request-scoped trust adapter, a connection authorization record, and transport admission checks. A small Rust library is a reasonable choice for policy parsing, signature verification, hostname-rule compilation, and immutable data structures. Keep certificate construction and validation inside the existing Gecko/NSS path. Gecko's documented Rust integration supports this implementation split, but the exact foreign-function interface (FFI) boundary must be reviewed and tested.[11]

The Personal Security Manager (PSM) asynchronous verification flow spans verification threads, the socket transport thread, and potentially parent/socket-process IPC. A policy pointer available only in the parent UI process is therefore insufficient. Capture a validated snapshot for each job and propagate a versioned result to the process that admits network traffic.[12] In this document, Mozilla's certificate TrustDomain is a verifier abstraction; it is unrelated to a SCONE Process Trust Domain.

Existing area	Proposed change	Validation focus
security/certverifier/CertVerifier.*	Carry immutable policy context through TLS verification	All validation branches and fallback attempts
security/certverifier/NSSCertDBTrustDomain.*	Scope trust eligibility and authenticated issuer inputs	Explicit distrust and alternate paths
mozpkix trust interface	Use existing trust and chain callbacks	Backtracking and chain ordering
security/manager/ssl/	Propagate result, enforce non-overridable denial, bind session state	IPC, resumption, overrides, concurrency
netwerk/protocol/http/	Gate new requests, coalescing, HTTP/3, and policy transitions	Actual application-data admission
Enterprise policy and browser UI	Bootstrap authority and show enforcement status	No content-controlled trust changes

[8] Mozilla Extension Workshop, Signing and distribution overview, accessed 10 September 2026. Signing and distribution rules for standard Firefox add-ons.

[9] Mozilla Firefox source, CertVerifier.cpp, moving branch inspected 10 September 2026. VerifySSLServerCert and its validation flow.

[10] Mozilla NSS mozpkix source, pkixtypes.h, moving branch inspected 10 September 2026. TrustLevel, TrustDomain, GetCertTrust, FindIssuer, and IsChainValid. See also NSSCertDBTrustDomain.h, interface reference; exact implementation requires pinned-tree review.

[11] Mozilla Firefox Source Docs, Including Rust Code in Firefox, accessed 10 September 2026. Gecko build integration for Rust components.

[12] Mozilla Firefox source, SSLServerCertVerification.cpp, moving branch inspected 10 September 2026. Asynchronous authentication and socket-process IPC.

These are source areas to resolve against the pinned tree, not a promise that a one-file patch is sufficient. The exact trust-cache, issuer-source, and thread-safety interactions inside NSSCertDBTrustDomain.cpp must be established on the pinned implementation. These are mandatory intake questions. The public interface and surrounding call sites establish feasibility, not implementation completeness.

Certificate acceptance algorithm

First derive the reference hostname from the actual requested origin using Gecko's URL and IDNA handling. Do not replace it with a CNAME target, an Alt-Svc endpoint, a proxy hostname, or an ECH outer public name. TLS service identity is based on the application's reference identifier rather than arbitrary intermediate discovery names.[13] A policy subtree is also different from a certificate wildcard: an scone.cloud subtree can protect deeper descendants, while leaf-certificate hostname matching still follows Firefox's normal rules.

Compile rules into a reverse-label trie. Include the apex explicitly, normalize one trailing dot according to the pinned URL behavior, and reject invalid labels, equal-specificity conflicts, and unauthorized public-suffix coverage. IP literals use separate exact-address rules or are outside the DNS policy. A request to an IP address must never be labelled as a verified scone.cloud service merely because it reaches the same machine.

For each TLS verification, capture the policy snapshot and effective rule. With no applicable rule, return the baseline verifier's result without adding policy-only anchors. With an enforcement rule, restrict which CA candidates can terminate the path. Preserve issuer discovery so an unapproved anchor candidate can still participate as an intermediate on a valid path to an approved anchor. In mozpkix terms, an otherwise trusted but unapproved anchor becomes InheritsTrust, not ActivelyDistrusted. Actual distrust retains priority.

Policy-supplied private anchors are an explicit additional trust source confined to authorized namespaces and TLS server authentication. An SPKI allowlist alone must not promote an arbitrary server-supplied certificate to an anchor. Validate supplied CA DER, match the digest, enforce CA role and usage restrictions, and retain embedded and administrative constraints. Where Firefox does not automatically enforce a desired trust-anchor validity or constraint rule, implement that check explicitly for the policy-supplied authority. Never claim that trusting a subordinate CA preserves its former parent's constraints automatically.

If a required intermediate is configured, check it during chain construction so the builder can try another path. mozpkix documents IsChainValid as a callback that can run before revocation is complete and can be invoked repeatedly; its chain array starts with the trust anchor.[10] A successful callback is therefore not a complete verification result. Publish authorization only after the entire verification succeeds, including the final chain postcondition and ordinary hostname checks. Normalize the final-chain representation at the adapter boundary instead of assuming every Firefox API uses the same ordering.

Conceptually, acceptance is the existence of a Firefox-valid path under the scoped trust sources whose terminal CA and optional intermediate satisfy the rule. It is not baseline verification followed by accepting any matching certificate found in the server's certificate bag. If only unapproved paths exist, deny; if an approved valid alternate path exists, attempt to construct it. Do not add an unrestricted fallback after the constrained attempt.

The result record should contain the canonical origin, policy identifier and digest, generation, rule digest, terminal SPKI, actual trust-source classification, relevant ordinary validation state, and an authorization expiry. Preserve origin attributes and existing isolation keys. Do not relabel a private root as built-in to make another

[13] IETF, RFC 9525 Service Identity in TLS, November 2023, sections 6.1 and 6.3. Reference identifiers and DNS identity matching.

check pass. A dedicated policy failure must remain non-overridable through saved exceptions, error pages, and automation paths in the managed product.

HTTP3 and alternative authentication paths

Firefox's HTTP/3 session implementation requests certificate authentication through PSM, but it also contains a WebTransport path based on application-supplied serverCertificateHashes and ECH fallback handling.[14] These are concrete reasons to inventory authentication paths instead of assuming that one verifier change covers all network APIs. For protected hosts, the first release should deny hash-only WebTransport authentication unless the same connection also receives native policy authorization. Ordinary PKI-authenticated WebTransport can be supported once it passes the same admission gate.

An ECH fallback check authenticates the fallback context; it must not become authorization to send protected-origin application data. Keep the requested origin and fallback verification purpose distinct in the result. Likewise, an HTTPS proxy's outer certificate authenticates the proxy, while the inner connection must satisfy the protected destination's rule. DNS HTTPS records and Alt-Svc may select a route; neither selects the authority policy for the destination.

Retain HTTP/3 for approved private PKI and reject unknown roots. Inventory any Firefox third-party-root transport heuristics on the pinned baseline and adapt only the necessary protocol-availability decision after valid authorization. Do not copy Chromium's known-root workaround into Firefox or bypass enterprise requirements for explicit proxy routing. HTTP/3 may remain unavailable where a mandatory proxy cannot support it; that is an operational constraint to report.

Protected origins must also reject cleartext HTTP and WS before transmitting credentials or request bodies; alternatively, perform a local HTTPS upgrade before any network request. A redirect from an unapproved cleartext endpoint is too late. Scope each protected origin's ports explicitly and evaluate each redirect destination independently. A redirect to an unmatched service retains ordinary Firefox semantics, so applications that require every dependency to be certified need a separate dependency allowlist.

A failed HTTP/3 attempt may retry over TCP only through the same policy-aware authorization path. Network failure can allow ordinary fallback. A rejected certificate cannot become acceptable because the browser chose another transport. Tests must record negotiated ALPN and server-side protocol; enabling HTTP/3 or receiving a successful page load does not prove that HTTP/3 ran.

Connections caches and update activation

The security boundary is application-data admission, not merely certificate verification. A connection verified under generation G may be selected after generation G+1 removes its CA. Before admitting a request or opening a stream, compare its origin and the connection authorization with the current policy. A generation check at verification completion is necessary but does not eliminate a later race at dispatch.

Define activation as an ordered transaction. Validate and persist the candidate, stop new admission for affected scopes, publish the new generation to participating processes, retire incompatible sessions and tickets, then acknowledge activation and resume authorized admission. A stale verification job must be restarted or rejected before it can authorize traffic. If a process cannot acknowledge the new generation, affected traffic remains blocked. Bytes already handed to the transport before the cutover cannot be recalled; record that limit explicitly.

---

[14] Mozilla Firefox source, Http3Session.cpp, moving branch inspected 10 September 2026. CallCertVerification, WebTransport certificate hashes, ECH fallback, and authentication completion. Exact branch behavior must be frozen in FDT-001 and FDT-003.

Use conservative reuse in the first release. Reuse within the same origin requires a live authorization record for the current effective rule. Cross-origin HTTP/2 and HTTP/3 coalescing involving a protected origin is disabled until candidate-origin revalidation is implemented and qualified. For the complete equivalent release, permit it only after normal Firefox checks and policy equivalence both succeed. Different child rules remain isolated even when a certificate's SANs cover both names.

Disable protected-origin TLS resumption initially, then add tickets bound to authorization and current policy. Reject tickets with absent metadata, removed anchors, stale epochs, expired certificates, or invalid authorization leases. Re-evaluate expiry and applicable revocation freshness even when the policy generation is unchanged. A policy digest alone is insufficient to keep an old authenticated session valid indefinitely.

Disable protected-origin 0-RTT in the first production profile and gate TLS False Start or any other application-data shortcut until policy authorization is complete. TLS 1.3 explicitly gives early data weaker replay properties.[15] This policy also avoids sending protected requests while relying on an obsolete authenticated session. Preserve normal early-data behavior for genuinely unmatched origins where reuse cannot cross into a protected scope.

Verifier caches, HTTP caches, service-worker responses, and back-forward cache entries have different roles. At minimum, no network operation may use obsolete connection authorization. For an origin newly protected or tightened, the strict browsing profile additionally prevents older cached documents and service workers from appearing as newly verified content. Evict or quarantine affected cache entries, close affected active documents, and prevent their background requests until fresh admission is possible. Site cookies and user data must not be silently erased as a substitute for correct enforcement. Full historical content provenance is a separate feature.

Signed policy and governance

Reuse the Chrome policy authority, governance model, and conformance corpus where implementations exist. Inspect the real repositories before claiming reuse. Preserve semantic compatibility for authorities, rules, revocations, and delegation, but do not assume Chrome version numbers or internal trust-source labels are portable. Add explicit format and required-feature negotiation for both browsers. A browser that cannot interpret a critical restriction must reject the candidate.

Use an immutable signed policy payload with a defined canonical representation, bounded sizes, no duplicate keys, explicit signature algorithms, and domain-separated signing. Separate CA issuance keys, policy-signing keys, browser-update keys, and recovery keys. The Update Framework provides a useful reference for signed metadata roles, version checks, expiry, and key rotation; adopting selected ideas does not make a custom protocol TUF compliant.[16]

The proposed bootstrap establishes the policy ID, authority keys, permitted update endpoints, and protected enrollment scope before browsing begins. It must retain a minimum protected scope even if no usable bundle is available. A corrupt or missing cache must not turn every hostname into an unmatched host. Preserve the last authenticated protected-scope inventory and require explicit signed authorization to release a scope from protection.

Policy condition	Required behavior
Valid unexpired snapshot	Enforce its rules and normal validation

[15] IETF, RFC 8446 The Transport Layer Security Protocol Version 1.3, August 2018, sections 2.3 and 8. Early-data and replay properties.

[16] The Update Framework, Specification, accessed 10 September 2026. Signed metadata, role separation, rollback checks, and expiry.

Policy condition	Required behavior
Candidate malformed or signature invalid	Retain the existing valid snapshot
Service unreachable	Retain the snapshot only until its permitted expiry
Snapshot expired	Block protected enrollment and retained scope; preserve unrelated browsing
Startup with missing or corrupt state	Block enrolled scope until authenticated recovery
Older signed generation replayed	Reject without resetting the persisted generation floor
Same generation with different payload	Reject as equivocation
Root-authorized recovery	Accept only through the specified recovery and epoch rules

The monotonic generation floor must survive ordinary browser restarts and profile recreation through managed enrollment. Plain profile storage cannot protect against an administrator restoring both policy and rollback state. That stronger attacker requires an external monotonic authority or hardware-backed state and belongs to a separately qualified threat model. Clock handling similarly needs a stated assumption: retain a last trusted time floor and use monotonic elapsed time during execution; significant rollback or uncertain expiry blocks protected traffic pending recovery.

Introduce old and new CA keys in one signed overlap policy, wait for a defined fraction of active clients to adopt it, switch service issuance, and remove the old key in a later generation. Offline clients do not authorize bypasses. Compromise recovery may skip overlap and revoke immediately, with availability consequences. CA possession proves control of a key, not permission to change browser trust; governance must authorize the update independently.

A child namespace rule may override its parent only through an explicitly authorized delegation, including allowed scope, maximum validity, allowed modes, and delegation depth. An unauthorized child must not weaken enforcement simply by being more specific. DNS control and proof of next-key possession are onboarding evidence, not complete authorization. Log policy changes and approvals without recording browsing history. Fleet health should report browser build, active policy generation, expiry, enforcement mode, and aggregate errors.

Policy lifetime, service-certificate lifetime, browser-release compatibility, and application A/B lifetime are separate dimensions. A certified application can remain on an approved older software slot while using a fresh certificate and policy. The browser policy must explicitly authorize any overlapping issuers; an application rollback must not roll back the browser's trust generation. No browser policy should infer current attestation solely from a three-month certificate lifetime.

Evaluation programme

The evaluation has four questions: does the design enforce the inherited trust property; where does a standard add-on fall short; what compatibility and performance costs result; and can the fork be maintained through browser and policy updates? Use stock Firefox, stock Firefox with the observation add-on, and the native fork built from the same baseline. Compare with the Chromium implementation only if a real build and evidence are available; otherwise compare requirements and fixtures, not invented measurements.

Generate a local test PKI with approved old/new roots, another normally trusted test root, a locally installed interception root, an unknown root, constrained subordinate CAs, cross-signed paths, and invalid leaves. A test

root models the trust role of a public CA but is not evidence about every production root-program behavior. Keep keys explicitly test-only and use reproducible scenario definitions, fixed seed fixtures where appropriate, and validity windows that remain controllable by the test clock.

Mozilla supplies xpcshell network-server infrastructure and PSM unit-test locations that can be extended.[17] Use real certificate validation throughout the security tests; convenience modes that disable certificate checking or automation's insecure-certificate acceptance invalidate the acceptance evidence. Mozilla also documents HTTP/2 and HTTP/3 Mochitest server options and limitations, so protocol-specific skip lists must be reported rather than treated as passes.[18]

Experiment	Main question and method	Primary result
FDE-01	Switch an authorized test endpoint to an unapproved but normally trusted CA; send synthetic cookie, header, and POST canaries	Server-observed application bytes and event timing for stock, add-on, and native
FDE-02	Replay all inherited C-01 through C-50 cases with Firefox-specific expected errors	Pass, fail, blocked, or not run for each case and transport
FDE-03	Reorder alternate and cross-signed paths; compare post-check-only control with scoped path construction	Correct path selection and false rejections
FDE-04	Exercise H1, H2, H3, fallback, proxy, ECH, WebSocket, and both WebTransport authentication modes	Per-path coverage and actual negotiated protocol
FDE-05	Update policy during handshakes, stream opening, reuse, and resumption	Admissions after the defined cutover and session retirement latency
FDE-06	Crash, restart, corrupt storage, replay policies, expire snapshots, and roll back clocks	Persistence and fail-closed behavior within the declared threat model
FDE-07	Test private roots through each qualified OS trust integration	Unmatched acceptance and protected rejection, including H3 availability
FDE-08	Vary rules, anchors, concurrency, cache state, and browser workload	Lookup, handshake, page-load, CPU, memory, and throughput overhead
FDE-09	Run CA overlap, emergency revocation, delegation, offline recovery, and application A/B scenarios	Adoption, cutover, recovery, and uninterrupted authorized use
FDE-10	Rebase onto two subsequent upstream security updates	Patch conflicts, review effort, regression count, and release delay
FDE-11	Exercise subresources, workers, downloads, private windows, containers, startup, and stored content	Request coverage and honest verification UI
FDE-12	Fuzz policy/IPC/chain boundaries and inject fail-open mutations	Findings and detection of seeded violations

For FDE-01, make the synthetic credentials valid only inside the local test. First show that the unmodified control reaches the server. For the add-on, measure exactly which canaries arrive before the callback; streamed bodies may be only partially sent. For native enforcement, the success criterion is zero protected HTTP request bytes and zero application effects at the rejected endpoint. TLS handshakes, DNS, and routing

[17] Mozilla Firefox Source Docs, Node HTTP Servers for Testing, and Mozilla source, PSM unit tests, accessed 10 September 2026. Native testing integration and test infrastructure.

[18] Mozilla Firefox Source Docs, Mochitest with HTTP2 and HTTP3 Support, accessed 10 September 2026. Protocol test flags and harness limitations.

metadata are outside that byte claim. Packet traces alone cannot prove absence of plaintext application data inside encrypted records; correlate instrumented server logs, connection IDs, and browser events.

For race tests, record separate timestamps for policy receipt, validation, persistence, admission freeze, published generation, process acknowledgments, and activation completion. Count only bytes admitted after the defined enforcement boundary as a cutover violation. Include forced thread interleavings and blocked IPC, rather than relying solely on probabilistic stress. Record new-rule activation, anchor removal, expiry with no generation change, and certificate expiry while connections remain open.

For performance, use identical hardware, OS image, browser commit, optimization flags, trust store, server, and network shaping. Randomize paired control and treatment order. Separate cold handshakes from warm reuse, H2 from H3, and 0-RTT-disabled protected traffic from ordinary traffic. Test 0, 10, 1,000, and 10,000 rules, small and large anchor sets, and shallow versus deep names. Report medians, p95, p99, confidence intervals, allocations, and raw sample counts. Begin with 30 independent process runs per main scenario and extend if uncertainty prevents judging the agreed budget.

Mozilla's Raptor catalogue includes browser and protocol performance workloads that can support this measurement programme.[19] A practical initial proposal is at most 2 percent median and 5 percent p95 page-load regression for unmatched browsing, and at most 2 milliseconds p95 additional local verification time for matched full handshakes on the designated reference machine. These are proposed engineering targets. Measure network-dependent revocation separately; report update-stall and session-retirement latency without inventing an "instant" guarantee. Correctness gates are mandatory even if these performance targets are met.

Backlog structure and execution

The accompanying backlog contains 48 FDT work items, dependency edges, proposed implementation locations, acceptance criteria, tests, expected evidence, and prompts suitable for Codex. All items are planned; no source repository has been inspected for implementation status. Logical repository owners must be resolved before editing. Reuse existing policy and controller implementations where verified, and avoid duplicating the Chrome programme under new names.

Milestone	Outcome	Gate
M0	Frozen baseline and threat model	Source call graph, supported platforms, and recovered Chrome traceability
M1	Observation add-on and leakage experiment	Documented add-on limits with actual server evidence
M2	Static native certificate enforcement	Scoped path building, no bypass, and normal validation preserved
M3	Complete network admission	H3, alternative paths, reuse, expiry, and cache transitions qualified
M4	Governed dynamic policy	Atomic activation, restart safety, CA rollover, and scoped delegation
M5	Product evaluation and maintenance	Linux/macOS packages, complete evidence, performance, and rebase exercise

[19] Mozilla Firefox Source Docs, Raptor, accessed 10 September 2026. Browser and HTTP2/HTTP3 performance workloads.

M1 is not a security-equivalent product release. M2 can run internal experiments with coalescing and protected resumption disabled. Full-equivalence claims require the relevant M3 through M5 gates, including inherited private-PKI HTTP/3 and same-policy coalescing cases. A platform or network surface without evidence remains unqualified or explicitly disabled for protected traffic; it is not silently covered by another platform's tests.

Unit and property tests should cover canonicalization, signatures, delegation, snapshot state, and trust-level translation. Gecko integration tests must exercise actual certificate construction and application-data admission. Browser tests cover UI, permissions, startup, private contexts, service workers, and caches. CI should run a focused blocking suite for each patch, a scheduled real H1/H2/H3 matrix, and full platform and update qualification for a release candidate. Report unavailable runners as blocked.

Build and release engineering must include a clean upstream import, a reviewable patch series, pinned dependencies, source and patch digests, an SBOM, and authenticated updates. Use Mozilla's supported setup flow to establish full native builds; artifact builds cannot validate the C++ verifier changes.[20] Compare reproducibility of unsigned payloads separately from signatures and notarization. Distinct product identity, profile locations, and update channels reduce accidental use of stock Firefox when enforcement is expected.

Each completed task must record actual commits, changed tests, commands, exit status, environment, and evidence digests. A passing mock is not browser enforcement evidence. Browser source review and an independent security assessment remain release gates; automation cannot supply their approval merely by creating a checklist. The evidence JSON begins with null measurements and `not_run` results to make this distinction executable.

Open decisions and residual risk

The principal unresolved implementation questions are the pinned trust-cache semantics, the exact transport-admission synchronization points, policy effects on ECH fallback, the scope of Firefox's private-root H3 behavior, and comprehensive cache/document invalidation. These are bounded source and experiment tasks in M0 through M3. They must be answered before promising a portable add-on or a small fixed patch count.

Central CA trust does not prove service freshness; policy expiry does not revoke bytes already disclosed; and local storage cannot defeat a privileged rollback of the entire endpoint. The fork also creates an ongoing browser-security maintenance obligation. Two successful rebases are a useful feasibility measurement, not proof of indefinite maintainability. Assign continuing ownership and a security-update release objective before deploying to users.

The recommended first executable slice is the local test PKI, the observation add-on timing experiment, and one native protected-origin H1/H2 test under a static policy. It establishes the key security difference quickly. Progress to HTTP/3 and dynamic policy only through the same connection-bound authorization contract, retaining the Chrome requirements as the acceptance baseline.

[20] Mozilla Firefox Source Docs, Getting Set Up To Work On The Firefox Codebase, accessed 10 September 2026. Native build setup entry point; exact commands and requirements are pinned during intake.

Sources

1. Earlier project reports, 26 July 2026: Extended_Chromium_Custom_Verifier_Companion_Whitepaper.docx, version 1.0, sections 1, 5–10 and Appendices C–D; and Constrained_Certificate_Trust_in_Chrome_Technical_Whitepaper_v2.1.docx. Retrieved project documents; no public URL. The companion is the normative compatibility baseline.

2. IETF, RFC 5280 Internet X.509 Public Key Infrastructure Certificate and CRL Profile, May 2008, sections 4.2.1.10 and 6. Trust anchors are validation inputs; anchor-specific checks must be explicit.

3. Mozilla MDN, webRequest.getSecurityInfo, modified 17 July 2025; accessed 10 September 2026. Callback timing, permissions, and chain option.

4. Mozilla MDN, webRequest.onHeadersReceived, accessed 10 September 2026. Response-stage event semantics.

5. Mozilla MDN, webRequest.CertificateInfo, modified 17 July 2025; accessed 10 September 2026. SPKI digests and certificate fingerprints.

6. Mozilla MDN, webRequest, accessed 10 September 2026. API trust boundary, startup, speculative requests, and host permissions.

7. Mozilla, Policy Templates for Firefox, accessed 10 September 2026. Certificates, ExtensionSettings, and DisableSecurityBypass sections.

8. Mozilla Extension Workshop, Signing and distribution overview, accessed 10 September 2026. Signing and distribution rules for standard Firefox add-ons.

9. Mozilla Firefox source, CertVerifier.cpp, moving branch inspected 10 September 2026. VerifySSLServerCert and its validation flow.

10. Mozilla NSS mozpkix source, pkixtypes.h, moving branch inspected 10 September 2026. TrustLevel, TrustDomain, GetCertTrust, FindIssuer, and IsChainValid. See also NSSCertDBTrustDomain.h, interface reference; exact implementation requires pinned-tree review.

11. Mozilla Firefox Source Docs, Including Rust Code in Firefox, accessed 10 September 2026. Gecko build integration for Rust components.

12. Mozilla Firefox source, SSLServerCertVerification.cpp, moving branch inspected 10 September 2026. Asynchronous authentication and socket-process IPC.

13. IETF, RFC 9525 Service Identity in TLS, November 2023, sections 6.1 and 6.3. Reference identifiers and DNS identity matching.

14. Mozilla Firefox source, Http3Session.cpp, moving branch inspected 10 September 2026. CallCertVerification, WebTransport certificate hashes, ECH fallback, and authentication completion. Exact branch behavior must be frozen in FDT-001 and FDT-003.

15. IETF, RFC 8446 The Transport Layer Security Protocol Version 1.3, August 2018, sections 2.3 and 8. Early-data and replay properties.

16. The Update Framework, Specification, accessed 10 September 2026. Signed metadata, role separation, rollback checks, and expiry.

17. Mozilla Firefox Source Docs, Node HTTP Servers for Testing, and Mozilla source, PSM unit tests, accessed 10 September 2026. Native testing integration and test infrastructure.

18. Mozilla Firefox Source Docs, Mochitest with HTTP2 and HTTP3 Support, accessed 10 September 2026. Protocol test flags and harness limitations.

19. Mozilla Firefox Source Docs, Raptor, accessed 10 September 2026. Browser and HTTP2/HTTP3 performance workloads.

20. Mozilla Firefox Source Docs, Getting Set Up To Work On The Firefox Codebase, accessed 10 September 2026. Native build setup entry point; exact commands and requirements are pinned during intake.